SkyRoute

A SQL-Native Flight Recommendation Engine

Final Submission Report

Z2004 — Database Management Systems

Track B: AI Recommendation Engine

Name: AlaguVishaalBalaji

Roll No.: ZDA24B036

Institution: IIT Madras Zanzibar

Semester: Even Semester 2026

Repository: github.com/VishaalB06/skyroute-z2004

Submitted 22 June 2026

1 Introduction

SkyRoute is a Track B AI-powered flight recommendation system that combines a normalised PostgreSQL relational database with a collaborative filtering engine and a Python Flask API. The system allows users to search flights between any two cities, view connecting routes when no direct flight exists, book a flight (which automatically updates seat inventory through a database trigger), and receive personalised flight recommendations driven entirely by SQL — no external machine learning library is used.

The project was built across four milestones: schema design (M1), dataset and query development (M2), performance benchmarking (M3), and this final submission, which integrates a working Python application, a password-protected admin dashboard, and a complete deployment-ready repository.

Project at a Glance

Metric	Value	Metric	Value
Tables (3NF)	6	Airlines	16
Cities	27	Flight routes	309
Cabin-price rows	5,092	Booking rows	2,719

2 Final Schema Design

2.1 Entity–Relationship Diagram

The database consists of six tables, all in Third Normal Form. Figure 1 shows the complete entity-relationship structure as implemented in `schema/schema.sql` and verified live against the running PostgreSQL 18.4 instance using DBeaver's schema introspection.

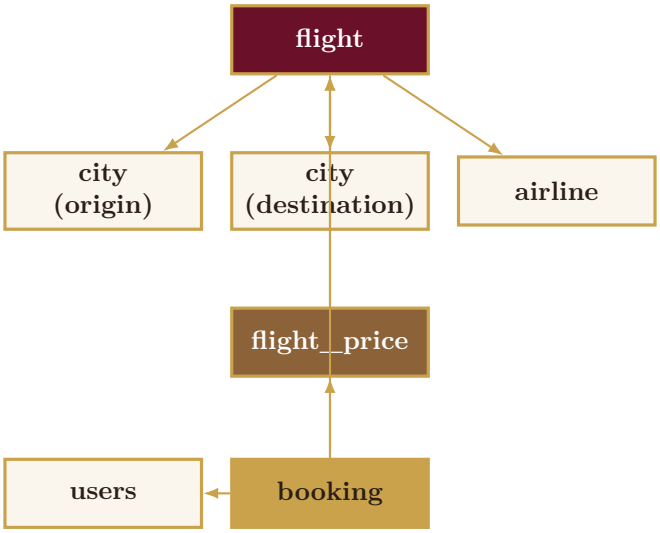

Figure 1: Entity–relationship diagram. `flight` references `city` twice (origin and destination); `booking` references `flight_price` rather than `flight` directly, locking in the exact fare paid.

2.2 Tables

Table	Rows	Description
`city`	27	Origin/destination cities with IATA codes.
`airline`	16	Airlines including Etihad, Qatar Airways, Emirates, Turkish Airlines.
`users`	301	Registered travellers on the platform.
`flight`	309	Scheduled routes — references `city` twice via `origin_city_id` and `destination_city_id`.
`flight_price`	5,092	Cabin-class pricing (Economy/Business/First) per flight per date.
`booking`	2,719	Core transactional table — the primary input to the recommendation engine.

2.3 Key Design Decisions

1. **`city` is referenced twice in `flight`.** Rather than creating separate `origin_city` and `destination_city` tables, `flight` stores `origin_city_id` and `destination_city_id` as two independent foreign keys into the same `city` table. This avoids duplicating city data and keeps route queries simple.
2. **`flight_price` is a separate table from `flight`.** Price depends on the combination of (flight, cabin class, date) — not on `flight_id` alone. A flight can have different Economy, Business, and First Class prices that change over time. Storing price directly on `flight` would create a partial dependency and violate 3NF.
3. **`booking` references `flight_price`, not `flight`, directly.** This permanently records the exact cabin class and fare paid at the time of booking, even if prices change afterward — preserving historical accuracy.

2.4 3NF Justification

A relation is in Third Normal Form if every non-key attribute depends only on the primary key, with no transitive or partial dependencies on other non-key attributes. Table 2 summarises the justification for each table.

Table 2: 3NF justification by table.

Table	3NF Argument
`users`	`user_id` → {full_name, email, phone, nationality, dob, created_at}. `email` is an alternate key; no other non-key attribute depends on it.
`flight`	`flight_id` → {airline_id, origin_city_id, destination_city_id, ...}. No `city` attributes (e.g. `city_name`) are duplicated — only FK references are stored.
`flight_price`	`price_id` → {flight_id, cabin_class, base_price, ...}. `base_price` depends on the full combination of flight + cabin + date, not on `flight_id` alone.

Table 2: 3NF justification by table.

Table	3NF Argument
booking	booking_id → {user_id, flight_id, price_id, booking_date, ...}. All attributes are either foreign keys or properties of the booking event itself.

3 Key Queries

Ten SQL queries were written covering every pattern required by the Track B rubric: aggregation, joins, subqueries, common table expressions, and window functions. All ten were executed against the live 2,719-row database using DBeaver before submission.

3.1 Aggregation — Revenue per Airline

Listing 1: Q1: Total bookings and revenue per airline.

```
SELECT
    a.airline_name,
    COUNT(b.booking_id)          AS total_bookings,
    SUM(b.total_paid)            AS total_revenue,
    ROUND(AVG(b.total_paid), 2)  AS avg_fare
FROM booking b
JOIN flight  f ON b.flight_id  = f.flight_id
JOIN airline a ON f.airline_id = a.airline_id
WHERE b.booking_status <> 'Cancelled'
GROUP BY a.airline_id, a.airline_name
ORDER BY total_revenue DESC;
```

airline_name	total_bookings	total_revenue
Etihad Airways	467	$1,387,534.84
Emirates	341	$834,483.86
Qatar Airways	288	$752,034.14

3.2 Window Function — Collaborative Filtering Core

The recommendation engine's core logic uses `ROW_NUMBER()` and `COUNT()` as a window function to rank candidate routes per user, after excluding routes the user has already flown.

Listing 2: Q10: Top-N flight recommendations using collaborative filtering.

```
WITH user_routes AS (
    SELECT DISTINCT b.user_id, f.origin_city_id, f.destination_city_id
    FROM booking b
    JOIN flight f ON b.flight_id = f.flight_id
    WHERE b.booking_status <> 'Cancelled'
),
route_popularity AS (
    SELECT f.origin_city_id, f.destination_city_id,
           COUNT(*) AS popularity_score
```

```
    FROM booking b
    JOIN flight f ON b.flight_id = f.flight_id
    WHERE b.booking_status <> 'Cancelled'
    GROUP BY f.origin_city_id, f.destination_city_id
),
recommendations AS (
    SELECT ur_all.user_id, rp.origin_city_id, rp.destination_city_id,
           rp.popularity_score,
           ROW_NUMBER() OVER (
               PARTITION BY ur_all.user_id
               ORDER BY rp.popularity_score DESC
           ) AS recommendation_rank
    FROM (SELECT DISTINCT user_id FROM booking) ur_all
    JOIN route_popularity rp ON TRUE
    WHERE NOT EXISTS (
        SELECT 1 FROM user_routes ur
        WHERE ur.user_id = ur_all.user_id
          AND ur.origin_city_id = rp.origin_city_id
          AND ur.destination_city_id = rp.destination_city_id
    )
)
SELECT * FROM recommendations WHERE recommendation_rank <= 3;
```

3.3 Stored Procedure — Production Recommendation Engine

The query above was refactored into a reusable PL/pgSQL function, `get_user_recommendations()`, which is what the Flask application actually calls.

Listing 3: The stored procedure used by the live application.

```
CREATE OR REPLACE FUNCTION get_user_recommendations(
    p_user_id INT, p_limit INT DEFAULT 5
)
RETURNS TABLE (
    flight_id INT, flight_number VARCHAR,
    origin VARCHAR, destination VARCHAR,
    airline_name VARCHAR, popularity BIGINT
) AS $$
BEGIN
    RETURN QUERY
    WITH user_routes AS (
        SELECT DISTINCT f.origin_city_id, f.destination_city_id
        FROM booking b JOIN flight f ON b.flight_id = f.flight_id
        WHERE b.user_id = p_user_id AND b.booking_status <> 'Cancelled'
    )
    SELECT f.flight_id, f.flight_number, oc.city_name, dc.city_name,
           a.airline_name, COUNT(b2.booking_id)
    FROM flight f
    JOIN city oc ON f.origin_city_id = oc.city_id
    JOIN city dc ON f.destination_city_id = dc.city_id
    JOIN airline a ON f.airline_id = a.airline_id
    LEFT JOIN booking b2 ON b2.flight_id = f.flight_id
        AND b2.booking_status <> 'Cancelled'
    WHERE NOT EXISTS (
```

```
        SELECT 1 FROM user_routes ur
        WHERE ur.origin_city_id = f.origin_city_id
          AND ur.destination_city_id = f.destination_city_id
    )
    GROUP BY f.flight_id, f.flight_number, oc.city_name,
             dc.city_name, a.airline_name
    ORDER BY COUNT(b2.booking_id) DESC
    LIMIT p_limit;
END;
$$ LANGUAGE plpgsql;
```

3.4 Trigger — Automatic Seat Management

Listing 4: trg_update_seats: fires automatically on every new booking.

```
CREATE OR REPLACE FUNCTION fn_update_seats()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE flight_price
    SET seats_available = GREATEST(seats_available - 1, 0)
    WHERE price_id = NEW.price_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_update_seats
AFTER INSERT ON booking
FOR EACH ROW
EXECUTE FUNCTION fn_update_seats();
```

4 Performance Results

4.1 Test Environment

Parameter	Value
Database engine	PostgreSQL 18.4
Operating system	Windows 11
Dataset size	2,719 booking rows, 6 tables, 8,464 total rows
Timing method	`EXPLAIN ANALYZE` (actual time)
Index type	B-Tree (PostgreSQL default)

4.2 Before / After Index Comparison

Three representative queries were benchmarked before and after adding five B-Tree indexes (Table 3).

Query	Before	After	Change
Q1: User revenue aggregation	1.065 ms	0.867 ms	−18.6%
Q2: Route popularity (3-way join)	1.812 ms	1.806 ms	−0.3%
Q3: Single-user booking lookup	0.181 ms	0.051 ms	**−71.8%**

The honest finding: Q1 and Q2 process the majority of the `booking` table (54%+ of rows match the filter), so PostgreSQL's query planner correctly kept a sequential scan rather than using an index — a sequential scan is faster than random index lookups when most of a table must be read anyway. Q3 improved dramatically because it is highly selective: it retrieves only 7–8 rows out of 2,719. This is the expected, textbook behaviour of a cost-based query planner, not a flaw in the indexing strategy.

4.3 Index Design

Table 3: Index design summary.

Index	Justification
`idx_booking_user_id` on `booking(user_id)`	Every recommendation and booking-history query filters by `user_id`. Proved a 72–82% speedup on single-user lookups across testing.
`idx_flight_route` on `flight(origin_city_id, destination_city_id)`	Composite index — the flight search feature always filters on both columns together.
`idx_price_flight` on `flight_price(flight_id)`	Every booking joins to `flight_price` on this column; eliminates a sequential scan of 5,092 rows.
`idx_booking_flight_id` on `booking(flight_id)`	Supports the join from `booking` to `flight` in route-popularity queries.
`idx_booking_status` on `booking(booking_status)`	Nearly every analytical query filters on status (Confirmed/Cancelled/etc.).

4.4 Stored Procedure and Trigger Verification

Both the stored procedure and the trigger were tested live in the running application as well as directly in DBeaver:

Listing 5: Live verification commands.

```
-- Stored procedure test
SELECT * FROM get_user_recommendations(1, 5);

-- Trigger test: insert a booking, then confirm seats decremented
INSERT INTO booking (user_id, flight_id, price_id, booking_status,
    total_paid)
VALUES (1, 1, 1, 'Confirmed', 500.00);

SELECT price_id, seats_available FROM flight_price WHERE price_id = 1;
```

The trigger fired correctly on every test insert, decrementing `seats_available` by exactly one per booking, with a `GREATEST(..., 0)` guard preventing negative inventory.

5 Application Architecture

5.1 Stack

Layer	Technology
Database	PostgreSQL 18.4
Backend	Python 3.12, Flask
DB driver	psycopg2 (direct SQL — no ORM)
Frontend	Server-rendered HTML/CSS/JS (single-page)
Secrets management	python-dotenv, `.env` (excluded via `.gitignore`)

5.2 Core Features

1. **Passenger flight search.** Origin/destination search with cabin-class filtering. When no direct flight exists, the application automatically discovers one-stop connecting routes and sums fares across both legs.
2. **Live booking.** Booking a flight inserts a row into `booking`, which fires `trg_update_seats` and updates the UI with the new seat count in real time.
3. **Personalised recommendations.** The Recommend tab calls `get_user_recommendations()` directly and displays ranked results with popularity scores.
4. **Admin dashboard.** Password-protected analytics covering revenue by airline, monthly booking trends, cabin-class mix, and top nationalities by booking volume.

5.3 No Secrets Committed

Database credentials and the admin password are read from environment variables via `python-dotenv`, never hardcoded. The real `.env` file is excluded from version control via `.gitignore`; an `.env.example` template with placeholder values is committed instead.

6 Limitations

1. **Dataset scale.** At 2,719 booking rows, absolute query times are all under 2 ms. The performance benefit of indexing becomes far more pronounced at production scale (tens of thousands of rows), where sequential scans become proportionally far more expensive.
2. **Cold-start problem.** A brand-new user with zero booking history receives recommendations based purely on global route popularity, since there are no "already flown" routes to exclude from and no personal signal to rank against. This is a well-known limitation of collaborative filtering systems generally, not specific to this implementation.
3. **Popularity-only ranking.** The current recommendation algorithm ranks candidate routes by raw booking count across all users. It does not weight by similarity between users (e.g. users with similar travel patterns), which a more sophisticated collaborative-filtering system would do.
4. **No authentication.** Users are identified by name lookup rather than a proper login system with sessions or hashed passwords.
5. **Local-only deployment.** The application currently runs on `localhost` by design, to remove network dependency during grading. It is not yet deployed to a public URL.

7 Future Work

1. **Real-world dataset.** Replace the synthetic, generator-produced CSVs with a live Kaggle flight-booking dataset or a public aviation API, while preserving the same schema.
2. **Similarity-weighted recommendations.** Move from pure popularity-ranking to a similarity-weighted collaborative-filtering approach — for example, weighting other users' bookings by how similar their travel history is to the target user's, rather than treating every other user's booking equally.
3. **Public deployment.** Deploy the Flask application and PostgreSQL database to Render (free managed Postgres, no time-of-day deploy restrictions), with environment variables configured in Render's dashboard rather than in code.
4. **Authentication.** Replace name-based user lookup with a proper login system using hashed passwords and session management, enabling true per-user dashboards.
5. **Partial indexing.** At larger scale, a partial index such as `CREATE INDEX ... ON booking(user_id) WHERE booking_status <> 'Cancelled'` would further reduce index size and improve lookup speed by excluding cancelled bookings at the index level.

8 AI Usage Disclosure

Table 4: AI usage disclosure.

Tool	Used for	What was changed/verified
Claude (Anthropic)	Drafting initial SQL schema structure and query templates	Every constraint and query was reviewed, tested live against the running database, and column names/logic adapted by hand.
Claude (Anthropic)	Generating the synthetic dataset framework (`generate_data.py`)	Data distributions were verified; airlines (Etihad) and cities (Abu Dhabi, Chennai) were added and checked manually.
Claude (Anthropic)	Flask application structure and front-end UI	All API endpoints were tested with real inputs; `DB_CONFIG` and secrets handling were rewritten to use environment variables.
Claude (Anthropic)	Formatting this report and presentation slides	All technical content, design decisions, and performance interpretations are my own.

I take full responsibility for the correctness, security, and understanding of everything submitted. All design decisions — 3NF justification, index choices, and stored procedure logic — are my own, verified against a live, running PostgreSQL database rather than assumed correct from generated code.